

FASTAPI + STREAMLIT RUNTIME

AI Engineer Ready · Week 7 — Two Processes, One Container

GOLDEN RULE FastAPI is internal-only (127.0.0.1:8000); Streamlit is the sole public port (\$PORT) and waits for /health before it starts.

1. Two processes, one container

FastAPI and Streamlit are two separate processes that happen to run inside the same Docker container. FastAPI binds to **127.0.0.1:8000** — internal-only, no external route exists to it at all. Streamlit binds to **0.0.0.0:\$PORT** (default 8080) and is the *only* public surface. The frontend reaches the backend over `http://localhost:8000` — a real loopback HTTP call, not a Python import.

	FastAPI (backend)	Streamlit (frontend)
Bind address	127.0.0.1:8000	0.0.0.0:\$PORT (default 8080)
Reachable from outside container?	No — internal only	Yes — the public URL
Local dev command	<code>uvicorn src.backend.main:app --host 0.0.0.0 --port 8000</code>	<code>python -m streamlit run src/frontend/app.py</code>
Local dev URL	<code>http://localhost:8000/docs</code>	<code>http://localhost:8501</code>

2. Health-gated startup order

`entrypoint.sh` enforces a strict order: start FastAPI, poll its `/health` endpoint, and only start Streamlit once FastAPI proves it's actually healthy — not just "process started." If Streamlit starts first and a user clicks a button before FastAPI is ready, the request fails with a connection error that looks like a bug but is really a race condition.

```
# entrypoint.sh (conceptual)
uvicorn src.backend.main:app --host 127.0.0.1 --port 8000 &
until curl -sf http://127.0.0.1:8000/health; do
sleep 1
done
streamlit run src/frontend/app.py --server.port=$PORT --server.address=0.0.0.0
```

3. Why /docs 404s on the live URL

FastAPI's Swagger UI at `/docs` is real and working — it just has no route from outside the container, because Streamlit is the only process with a public port. This is by design, not a bug: it keeps the raw API surface (and any internal endpoints) away from end users, who should only ever see the Streamlit UI.

■ Pitfall — Handling None from a backend call

A common real bug: the frontend calls the backend, gets back `None` (e.g. before ingestion has happened, or on a transient error), and the Streamlit code crashes trying to read a field off it. Always check the response before rendering — treat every backend call as something that can fail.

✓ \$PORT, not a hardcoded number

Streamlit reads `$PORT` instead of a fixed value because the platform decides the port at runtime (Render, Azure App Service, etc. all assign their own). Hardcoding 8080 works locally and silently breaks in the cloud.

4. Interview Prep

Q: Why run FastAPI and Streamlit as two processes instead of one?

A: Streamlit has no first-class way to expose a separate, documented HTTP API; FastAPI has no built-in UI. Two processes let each tool do what it's good at, connected over a normal HTTP boundary.

Q: What problem does the health-gated startup order solve?

A: It prevents a race condition where Streamlit accepts user clicks before FastAPI is ready to serve them, which would otherwise surface as intermittent, hard-to-reproduce connection errors.

Q: Why is FastAPI bound to 127.0.0.1 instead of 0.0.0.0?

A: Binding to loopback-only means there is no network path to it from outside the container at all — it's not just "not routed," it's unreachable, which is a stronger and simpler guarantee than firewall rules.